

浙江大学实验报告

专业: _____
姓名: _____
学号: _____
日期: 2024.10.21
地点: 东三 406

课程名称: 电路与电子技术实验 I 指导老师: 张德华 成绩: 95
实验名称: PWM 电机调速控制 同组学生姓名: _____

一、实验目的

通过此次实验,我将了解 PWM 电机调速控制的原理及应用、学习时序逻辑电路的简单设计与应用、数字式 PWM 电路的基本结构及其实现方式,从而为在前期智能小车的基础上增加调速功能,并在此过程中进一步熟悉基于原理图设计的、更加复杂的 VHDL 语言编程的流程、运用 Quartus 软件仿真数字电路、以及示波器等常用电子仪器的使用。

二、基本实验内容

(一) 应用 QUARTUS 软件设计、仿真 PWM 电路,基于 FPGA 开发板综合布线

1. 实验原理

1. 数字式 PWM 电路的基本结构如图 1 所示。其中,方波发生电路产生固定频率的方波信号,代表数字电路高电平 1 与低电平 0 的变化情况;N 进制计数器输出波形为锯齿波,大小随输入脉冲在 0~N-1 间循环变化;每检测到一个脉冲信号,计数器加 1,直到计数达到 N,则重新由 0 开始计数(如图 2);比较器 A 端输入来自计数器的锯齿波信号,B 端输入外部开关设定的设定值,则当 $A > B$,比较器输出高电平 1,反之输出低电平 0。故最终输出波形为方波(如图 2),且占空比可由设定值改变,从而实现 PWM 逐档调节。

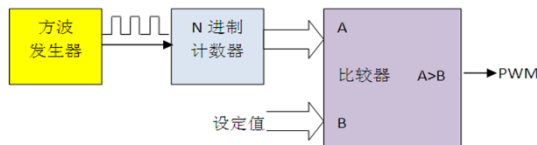


图 1

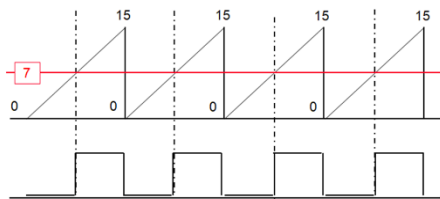


图 2

若方波发生器产生频率为 f 的方波,比较器 B 端设定值为 k ,则 N 进制计数器输出频率为 f/N 的锯齿波,比较器输出频率为 f/N 的方波,占空比为 k/N ,共可设置 N 档。

2. FPGA 实现 PWM 设计框图如图 3 所示。FPGA 芯片可输出固定频率为 50MHz 的方波信号,但由于该系统频率过高,不适合用于 PWM 调速电路中,故须在计数器前加入分频器,将 50MHz 转换为 PWM 要求的频率。另外,比较器设定值可由 FPGA 板上开关设定,无需反复编译并下载文件至 FPGA。

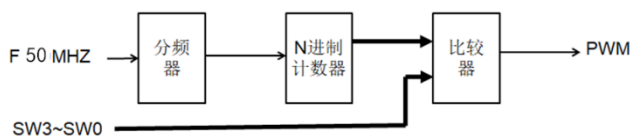


图 3

3. FPGA 50MHz 信号、GPIO 及 SW9~SW0、LEDR9~LEDR0 引脚如图 4~图 6 所示：

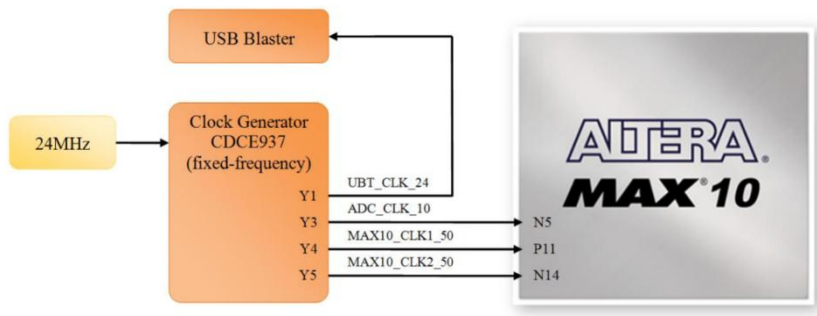


图 4



图 5

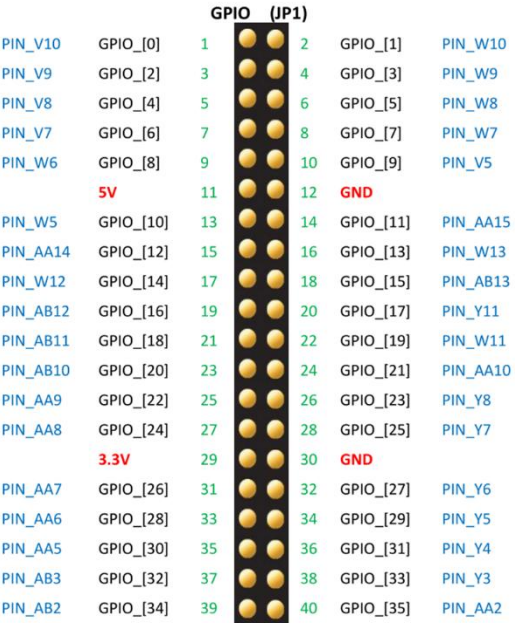


图 6

2. 主要仪器设备

Quartus 软件。

3. 测试方案与测试过程

按图 3 与实际测试需要，在 Quartus 中模块化编写程序如图 7~图 11 所示：

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5 entity trace1 is
6     port( LED6,LED7: in std_logic;
7           R,L:out std_logic;
8           clk0,rst: in std_logic;
9           PWM_in: in std_logic_vector(3 downto 0);
10          Div_set : in std_logic_vector (3 downto 0);
11          light: out std_logic_vector(3 downto 0);
12          PWM_out, PWM_out2: out std_logic);
13 end trace1;
14
15 architecture BEHAV of trace1 is
16
17     Component CNT16 is
18     port ( Clk,Clr_n: in std_logic;
19           Q: out std_logic_vector(3 downto 0);
20           CO:out std_logic);
21     end Component;
22
23     Component COMP4BIT is
24     port ( clk: in std_logic;
25           A,B: in std_logic_vector(3 downto 0);
26           AGB1,AGB2:out std_logic);
27     end Component;
28
29     Component DIVX is
30     port ( Freq_in,Clr_n : in std_logic;
31           Div_num:in natural range 0 to 49999999;
32           Freq_out: out std_logic);
33     end Component;
34
35     Component SET is
36     port ( Div: in std_logic_vector(3 downto 0);
37           num: out natural range 0 to 200000);
38     end Component;
39
40     signal D: std_logic_vector(1 downto 0);
41     signal clk_temp,clk_temp2: std_logic;
42     signal A_temp: std_logic_vector (3 downto 0);
43     shared variable Div_num_set : natural range 0 to 200000;
44
45     begin
46         U1: SET port map(Div=>Div_set, num=>Div_num_set);
47         U2: DIVX port map(Freq_in=>clk0, clr_n=>rst, Div_num=>Div_num_set, Freq_out=>clk_temp);
48         U3: CNT16 port map(clk=>clk_temp, clr_n=>rst, Q=>A_temp, CO=>clk_temp2);
49         U4: COMP4BIT port map(clk=>clk_temp2, A=>A_temp, B=>PWM_in, AGB1=>PWM_out, AGB2=>PWM_out2);
50
51         D<= LED7 & LED6;
52         light<=PWM_in;
53     process(D)
54     begin
55         case D is
56             when "00" => L<='0';R<='0';
57             when "01" => L<='1';R<='0';
58             when "10" => L<='0';R<='1';
59             when "11" => L<='1';R<='1';
60             when others => null;
61         end case;
62     end process;
63 end BEHAV;

```

图 7 顶层程序 trace1

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5 entity SET is
6     port(Div: in std_logic_vector(3 downto 0);
7           num: out natural range 0 to 200000);
8 end SET;
9
10 architecture BEHAV of SET is
11 begin
12     process(Div)
13     begin
14         case Div is
15             when "0000" => num <= 1;
16             when "0001" => num <= 100;
17             when "0010" => num <= 400;
18             when "0011" => num <= 800;
19             when "0100" => num <= 1200;
20             when "0101" => num <= 2000;
21             when "0110" => num <= 4000;
22             when "0111" => num <= 6000;
23             when "1000" => num <= 8000;
24             when "1001" => num <= 10000;
25             when "1010" => num <= 12500;
26             when "1011" => num <= 15000;
27             when "1100" => num <= 20000;
28             when "1101" => num <= 30000;
29             when "1110" => num <= 40000;
30             when "1111" => num <= 80000;
31         end case;
32     end process;
33 end BEHAV;

```

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5 entity CNT16 is
6     port(Clk,Clr_n: in std_logic;
7           Q: out std_logic_vector(3 downto 0);
8           CO: out std_logic);
9 end CNT16;
10
11 architecture BEHAV of CNT16 is
12     signal Q_tmp: std_logic_vector(3 downto 0);
13 begin
14     CO <= '1' when (Q_tmp = "1111") else '0';
15     process(Clk,Clr_n)
16     begin
17         if (Clr_n = '0') then
18             Q_tmp <= "0000";
19         elsif (Clk 'event and Clk = '1') then
20             Q_tmp <= Q_tmp + 1;
21         end if;
22     end process;
23     Q <= Q_tmp;
24 end BEHAV;

```

图 8 设定分频数程序 SET

图 10 16 进制计数器程序 CNT16

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5 entity DIVX is
6     port(Freq_in,Clr_n: in std_logic;
7           Div_num: in natural range 0 to 49999999;
8           Freq_out: out std_logic);
9 end DIVX;
10
11 architecture BEHAV of DIVX is
12 begin
13     process (Freq_in, Clr_n)
14         variable cnt_val: natural range 0 to 49999999;
15     begin
16         if (Clr_n = '0') then
17             cnt_val := 0;
18         elsif (Freq_in 'event and Freq_in = '1') then
19             if (cnt_val = Div_num - 1) then
20                 cnt_val := 0;
21             elsif (cnt_val < Div_num / 2) then
22                 cnt_val := cnt_val + 1;
23                 Freq_out <= '0';
24             else
25                 cnt_val := cnt_val + 1;
26                 Freq_out <= '1';
27             end if;
28         end if;
29     end process;
30 end BEHAV;

```

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5 entity COMP4BIT is
6     port(clk: in std_logic;
7           A,B: in std_logic_vector(3 downto 0);
8           AGB1,AGB2: out std_logic);
9 end COMP4BIT;
10
11 architecture BEHAV of COMP4BIT is
12     signal B_temp: std_logic_vector(3 downto 0);
13 begin
14     process (A,B,clk)
15     begin
16         if (clk 'event and clk = '1') then
17             B_temp <= B;
18         end if;
19         if (A >= B_temp) then
20             AGB1 <= '1';
21             AGB2 <= '1';
22         else
23             AGB1 <= '0';
24             AGB2 <= '0';
25         end if;
26     end process;
27 end BEHAV;

```

图 9 分频器程序 DIVX

图 11 比较器程序 COMP4BIT

表 1 简要列举了部分信号与变量的含义:

名称	含义
LED6, LED7, D	光电管检测输入。
CLK0	FPGA 系统频率 50MHz 输入。
RST	复位信号输入。用于程序初始化。
PWM_IN, LIGHT	比较器 B 端设定值输入。由 FPGA SW3~SW0 设定, 对应 LED

	指示开关情况，便于观察。
DIV_NUM	分频数输入。由 FPGA SW9~SW6 设定，避免反复的编译、调试与下载工作。
PWM_OUT, PWM_OUT2	PWM 信号输出。分别对应左右电机输入 ENA, ENB。
L, R	电机控制信号输出。分别对应左右电机输入 IN2, IN3。
CNT_VAL	分频器计数变量。原理与功能与计数器相似，但达到上界时无法自动归零。
CO	计数器进位输出。保证占空比在一个周期内稳定。
CLK_TEMP, CLK_TEMP2, A_TEMP	中间变量。用于在不同子程序间传递信号与变量。

表 1

编译通过后，分别新建 DIVX, CNT16, COMP4BIT 三个子程序的波形分析文件，合理设置输入波形及周期（默认占空比 50%），观察输出波形特征，并与理论逻辑关系进行比较，完成仿真工作。

之后结合图 4, 5, 6 及程序，在 Quartus 中锁定各输入输出量管脚，如图 12 所示。并生成后缀名为.pof 的下载文件，导入 FPGA 板中。

Node Name	Direction	Location	I/O Bank	REF Group	Termination	Location	Standby	Reserved	Retention	Strap	Slew Rate	Differential	Preserve
clk0	Input	PIN_P11	3	B3_N0		PIN_P11	2.5 V		12m...lt				
Div_set[3]	Input	PIN_F15	7	B7_N0		PIN_F15	2.5 V		12m...lt				
Div_set[2]	Input	PIN_B14	7	B7_N0		PIN_B14	2.5 V		12m...lt				
Div_set[1]	Input	PIN_A14	7	B7_N0		PIN_A14	2.5 V		12m...lt				
Div_set[0]	Input	PIN_A13	7	B7_N0		PIN_A13	2.5 V		12m...lt				
L	Output	PIN_Y6	3	B3_N0		PIN_Y6	2.5 V		12m...lt	2 (d...ult)			
LED6	Input	PIN_W9	3	B3_N0		PIN_W9	2.5 V		12m...lt				
LED7	Input	PIN_W10	3	B3_N0		PIN_W10	2.5 V		12m...lt				
light[3]	Output	PIN_B10	7	B7_N0		PIN_B10	2.5 V		12m...lt	2 (d...ult)			
light[2]	Output	PIN_A10	7	B7_N0		PIN_A10	2.5 V		12m...lt	2 (d...ult)			
light[1]	Output	PIN_A9	7	B7_N0		PIN_A9	2.5 V		12m...lt	2 (d...ult)			
light[0]	Output	PIN_A8	7	B7_N0		PIN_A8	2.5 V		12m...lt	2 (d...ult)			
PWM_in[3]	Input	PIN_C12	7	B7_N0		PIN_C12	2.5 V		12m...lt				
PWM_in[2]	Input	PIN_D12	7	B7_N0		PIN_D12	2.5 V		12m...lt				
PWM_in[1]	Input	PIN_C11	7	B7_N0		PIN_C11	2.5 V		12m...lt				
PWM_in[0]	Input	PIN_C10	7	B7_N0		PIN_C10	2.5 V		12m...lt				
PWM_out	Output	PIN_Y5	3	B3_N0		PIN_Y5	2.5 V		12m...lt	2 (d...ult)			
PWM_out2	Output	PIN_Y3	3	B3_N0		PIN_Y3	2.5 V		12m...lt	2 (d...ult)			
R	Output	PIN_Y4	3	B3_N0		PIN_Y4	2.5 V		12m...lt	2 (d...ult)			
rst	Input	PIN_Y7	3	B3_N0		PIN_Y7	2.5 V		12m...lt				

图 12

4. 结果记录与分析

以下为子程序逻辑仿真测试结果及分析：

1. DIVX. 设定 DIV_NUM=10, CLR_N 恒为高电平，FREQ_IN 周期 20ns，得到仿真波形如图 13 所示：

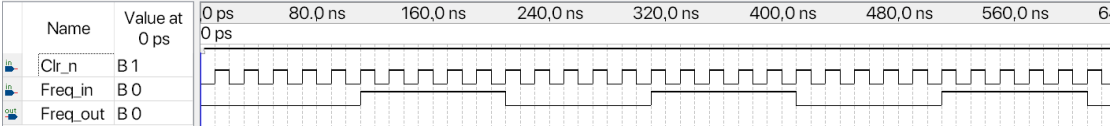


图 13

经检验，FREQ_OUT 周期 200ns。模块功能符合预期。

2. CNT16. 设定 CLR_N 恒为高电平，CLK 周期 20ns，得到仿真波形如图 14 所示：

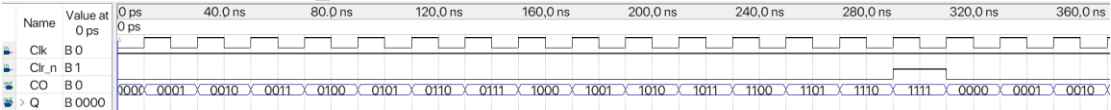


图 14

经检验，Q 在每个高电平上升沿处计数+1，CO 在 Q=1111 时为 1，其余为 0。模块功能符合预期。

3. COMP4BIT。设定 A 端每 5ns 计数+1（16 进制），B 端设定值为 8（1000），得到仿真波形如图 15 所示：

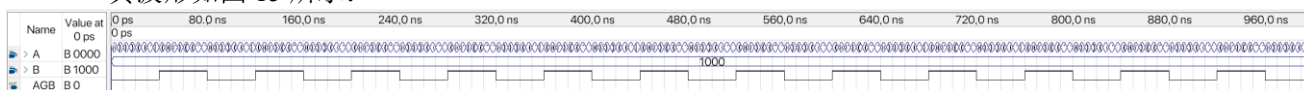


图 15

经检验，AGB 在 A 端计数为 0~7 时为 0，计数为 8~15 时为 1。模块功能符合预期。

（二）设定不同占空比，观察记录 PWM 的输出波形，测量周期、幅度和占空比

1. 实验原理

PWM 电路通过改变输出波形的占空比改变一个周期内高电平的宽度，从而实现调速效果，但并不改变输出波的周期与幅度。而占空比设定可通过 FPGA 板上的开关实现。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S、数字示波器 DSOX1102G

3. 测试方案

设定分频数 Div_num=2000。令稳压源直接为 FPGA 板提供+5VDC。并将 FPGA Y3 或 Y5 引脚（即程序中 PWM_OUT2 或 PWM_OUT）信号输入至示波器 CH1 通道。分别调节 FPGA 板上 SW3~SW0 至 0000、0100、1000、1100、1111，即设定占空比分别为 100%、75%、50%、25%、0%。在示波器 Meas 界面添加周期、幅度、占空比至底端栏中，观察不同占空比下 PWM 电路输出的各项参数。

4. 结果记录

不同占空比下，得到示波器截图如图 16~20 所示：

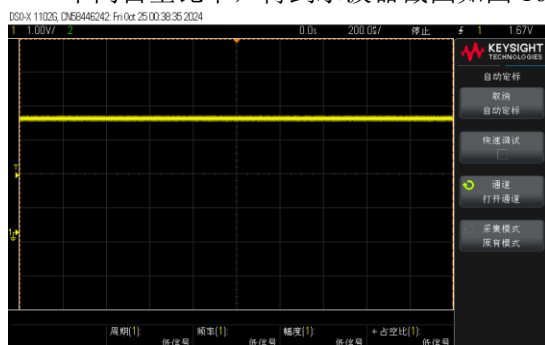


图 16 B=0000

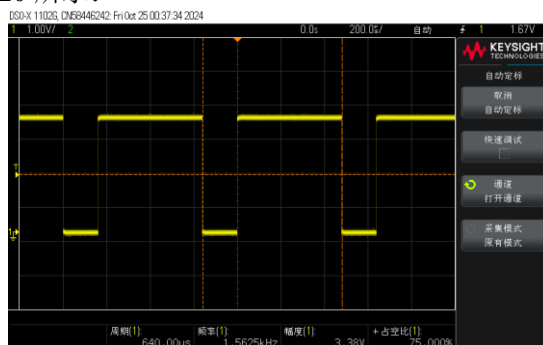


图 17 B=0010

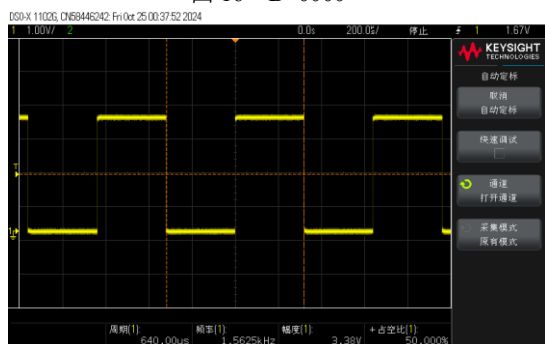


图 18 B=1000

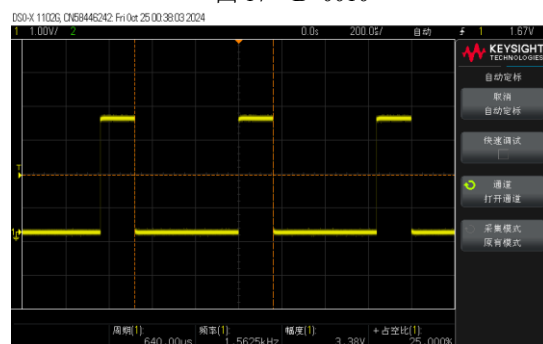


图 19 B=1100

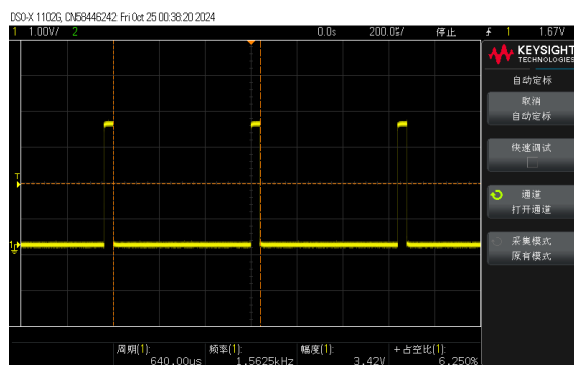


图 20 B=1111

5. 测试结果分析

该数字 PWM 电路中，输出波形为方波，周期 640.00us，幅度约 3.4V。理论上，由于此时 Div_num=2000，计数器为 16 进制，FPGA 系统频率 50MHz，故输出频率应为

$$\frac{50\text{MHz}}{2000} \times \frac{1}{16} = 1.5625\text{kHz}$$

即周期为 640us。比较可知实验正确。

可以注意到，改变设定挡位，仅会改变输出波的占空比，而不改变其周期、幅度。另外，当 B=1111 时，占空比并非 0%，而是 6.25%。这是由于图 11 所示红色虚线框中，A=B 时比较器仍可输出高电平 1。故当 A=1111 时，比较器输出 1，其占空比为 1/16=6.25%。

（三）PWM 输出至电机驱动模块，观察记录不同占空比时电机的运动速度

1. 实验原理

PWM 电路输出波形占空比越低，一个周期内有效值越小，电机运动速度越小。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S

3. 测试方案

设定分频数 Div_num=2000。设置稳压源输出+18VDC 至小车驱动板，并结合图 12、表 1 将 FPGA、驱动板、检测板各引脚连接。分别调节 FPGA 板上 SW3~SW0 至 0100、1000、1100，即设定占空比分别为 75%、50%、25%，观察并记录 1min 内小车轮胎转速。

4. 结果记录与分析

测试结果如表 2 所示：

占空比	1min 内圈数	转速 (r/s)
25%		
50%		
75%		

表 2

由表可知，PWM 电路输出波形占空比越低，一个周期内有效值越小，电机运动速度越小。

（四）设置三种不同分频频率，观察记录“欠调，正常，过调”电机工作情况

1. 实验原理

对于调频而言，欠调指调制信号的频率低于载波频率，而过调指调制信号的频率高于载波频率。前者会放大低频信号，压缩高频信号，可以减小信号失真和噪声，但会降低信号的带宽利用率；后者会截断高频信号，只有低频信号能通过传输，可以提高信号的带宽利用率，但会放大信号失真和噪声。

而在 PWM 调速电路中，选择合适的频率，占空比改变时有明显的速度变化，而欠调和过调变化均不明显，无法有效调速。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S、数字万用表 UT890D+

3. 测试方案

通过 SW9~SW6 调节分频数 Div_num 分别为 100、400、2000、6000、10000、40000、80000，由 $f=50\text{MHz}/\text{Div_num}$ 可计算得 PWM 频率如表所示。通过 SW3~SW0 设定 PWM 挡位分别为 0、2、4、8、12、15。分别用万用表电压档测量电机两端电压，由于电机转速与两端电压成正比关系，故可间接定性反映电机转动速度。

4. 结果记录

测试结果如表 3 所示：

挡位	电机电压（V）						
	500kHz	125kHz	25kHz	8.3 kHz	5kHz	1250Hz	625Hz
0							
2							
4							
8							
12							
15							

表 3

同时可以发现，PWM 输出频率越高，电机停转时发出的噪音越尖；而输出频率为 625Hz，挡位为 15 时，电机仍能转动，但发出较响的低沉运行噪声。

5. 测试结果分析

由表 3 可知，在 PWM 调速电路中，选择合适的频率，占空比改变时有明显的速度变化，而欠调和过调时，挡位间电机电压、转速变化均不明显，尤其是挡位<4 的区域，转速几乎没有差别，无法有效调速。猜测与电机内部的电感有关。

同时猜测电机运行噪声与 PWM 频率间存在一定关系，PWM 输出频率越高，电机停转时发出的噪音频率越高。频率很低时，电机发出较响的低沉运行噪声，猜测与电机启停时间宽度较大有关。

三、实验讨论与心得

1. 小车调速常用两种方式：直接调压法与 PWM 法。前者示意图如图 21 所示，利用电阻分压改变电机两端电压调速。但这种方式有一定缺陷：一是电机两端电压过低时，无法产生足够的扭矩使电机启动；二是该电路在调压电阻上消耗大量能量，产生大量热量不易散发，从而大大降低电效率，并造成电路，尤其是 FPGA 芯片运行不稳定。而后者示意图如图 22 所示，利用控制电路间歇导通和电机运动惯性调速。相比于直接调压法，PWM 控制导通时电机两端电压始终等于 E，能提供足够大的扭矩使电机启动；同时不必在调压电阻上消耗能量，耗能更小。

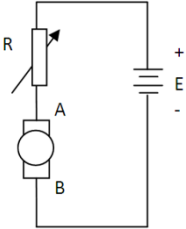


图 21

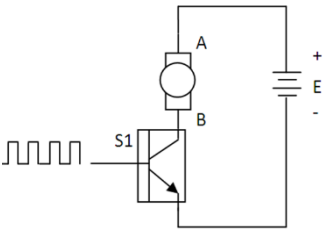


图 22

2. 实验（二）中，发现若令稳压源向驱动板供电+18VDC，驱动板再向 FPGA 供电，则测得 FPGA 输出波形不稳定（以图 23、24 为例，幅度、低电平基准等发生了很大的变化），同时测得驱动板向 FPGA 供电波形实为正弦波，而非稳定直流。猜测上述现象与驱动板内降压电路相关。

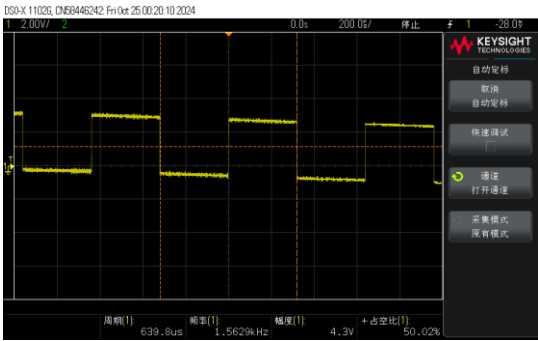


图 23

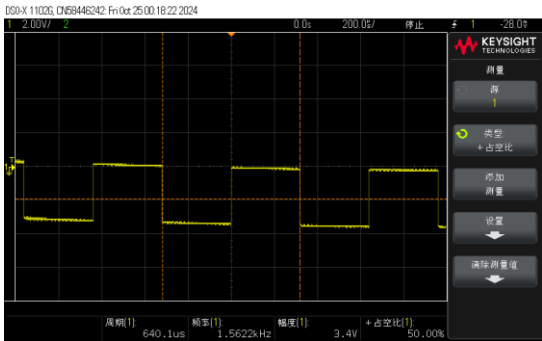


图 24

3. 观察主程序波形分析文件时，发现当 $B=1000$ 时，无论分频数 Div_num 设定为多少，初始时总有一段持续的高电平，宽度为 31/32 个周期。以图 25 为例，此时 $Div_num=10$ ，CLK0 周期 20ns，初值为 0，经 PWM 调制后周期为 $20ns \times 10 \times 16 = 3.2us$ 。分析初始高电平，除 0~10ns 低电平不计入，剩余部分宽度为 3.1us。猜测上述现象与设定值 B 与信号 CO 有关，但目前问题仍有待解决。

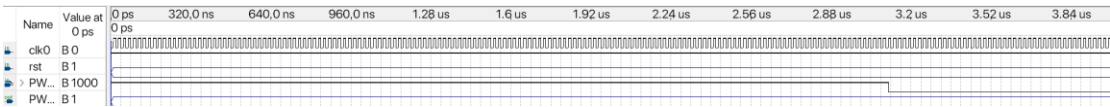


图 25

4. 此次实验中，采取了多个手段便利实验操作。例如引入 PWM 设定值指示灯 LIGHT，当对应开关打至 1 时灯亮，便于记录观察当前设定值；新建 SET 子程序，实现用开关 SW9~SW6 调节分频数 Div_num ，大大便利了实验（四）的操作，避免了因调试、编译、下载程序造成的大量时间消耗；实验（四）中利用万用表测量电机两端电压，间接反映电机转速，避免了在较长时间内观察记录电机转速的时间消耗。